

## Antigena-Inspired Self-Learning AI Defense System

Modern enterprises face fast-moving, sophisticated threats that evade signature-based defenses. We propose building an **AI-driven defense system** that “self-learns” normal behavior and automatically contains anomalies – much like Darktrace’s Antigena but using open methods. Our system will ingest diverse telemetry (network flows, logs, host events, identity/activity logs, cloud/SaaS APIs) and build dynamic behavioral models per device/user. In real time it scores deviations as anomalies and can trigger precise, automated responses (e.g. blocking malicious flows or quarantining endpoints) that preserve normal operations.

Importantly, we **do not replicate Darktrace’s proprietary algorithms** (e.g. their patented “Hyper-Cylinder” Bayesian model and “energy landscape” analytics). Instead we use well-known unsupervised ML (Isolation Forest, One-Class SVM, Autoencoders, etc.) to approximate the “pattern of life” approach. Like Darktrace, our goal is to detect **unknown / novel attacks by spotting deviations from baseline behavior**. We will define clear goals (what to include and omit), a concrete threat/use-case model, a data architecture, feature engineering, ML training and drift adaptation, scoring and explainability methods, a safe response engine, and a full deployment plan. We provide implementation checklists with code snippets, comparison tables (models, datasets, actions), mermaid diagrams (system architecture, timeline), and evaluation plans (metrics, red-team tests). Our focus is on technical feasibility and reproducibility (with open tools), not on proprietary performance claims.

Key distinctions: we **replicate** Antigena’s **core idea** – *self-learning anomaly detection + surgical response* – but **not its secret sauce**. For example, Darktrace describes Antigena as “surgically enforcing the normal ‘pattern of life’” of a compromised device, dynamically adapting response to threat severity. We will mimic this via data-driven baselining, not hard-coded rules. Darktrace’s whitepapers emphasize unsupervised Bayesian learning: “The system does not rely on knowledge of past attacks. Instead, it discovers previously unknown threats by detecting deviations from normal behavior”. We will similarly train on routine traffic to spot statistical anomalies.

In summary, **our project** will design and prototype an “Antigena-like” system (self-learning ML + automated response) step-by-step: from data ingestion, modeling, anomaly scoring, to safe action

workflows and continuous retraining. We will rigorously evaluate trade-offs (detection vs false positives) and chart out a realistic implementation timeline and architecture. The outcome is a detailed project report and guide to building such a system, suitable for enterprise use, with cited references from industry/academic sources.

## 1. Project Goals & Scope

- **Scope (include):** Unsupervised ML for anomaly detection in enterprise logs/flows; real-time monitoring; alerting and (optionally) automated containment actions. We aim to cover *network traffic, host telemetry, cloud/SaaS logs, identity/access logs*, etc., much like a full-stack AI defense platform. The system will continuously update its “normal” model and handle concept drift. We will implement core ML models (e.g. Isolation Forest, Autoencoders) and a response engine (block/alert mechanisms).
- **Constraints (exclusions):** No reproduction of Darktrace’s *proprietary* algorithms or code. Darktrace’s research uses advanced “Recursive Bayesian Estimation” and matrix methods (their patent calls it a “Hyper-Cylinder” Bayesian time-series model). We will **not** have access to these, so we use standard unsupervised techniques. We also will not have Darktrace’s massive training data or specialized hardware. We accept that our prototype will be a proof-of-concept, not a product. We will *note* the limitations this imposes.

### Objectives:

- **Self-Learning Detection:** Build an ML pipeline that ingests raw telemetry and learns normal behavior without human labels. It should flag statistically unusual events. (As Darktrace notes, “unsupervised learning captures the full spectrum of threat” without requiring labels.)
- **Anomaly Scoring & Explainability:** Produce a continuous anomaly score per device/user/event. Provide some interpretable features (for example, why a score is high) to build trust.
- **Automated Response:** Define containment actions (e.g. block IP, throttle connections, quarantine host) that the system can execute automatically or with minimal SOC oversight. Include safety checks and escalation (e.g. human confirmation for severe actions). Darktrace’s examples (block matching flows, enforce “pattern of life”, isolate device) will guide this.

- **Integration & Platform:** Design architecture (data collectors, databases, ML engine, APIs, dashboard) to operate in (unspecified-sized) enterprise networks. We assume a typical medium-size enterprise (thousands of devices), but design to scale.
- **Evaluation:** Define metrics (precision, recall, AUC, false-positive rate) and testing plan (simulations, red-team attacks). Prioritize metrics from academic/industry sources.
- **What we will *not* replicate:** Aside from proprietary algorithms, we will not try to clone Darktrace's AI Analyst (an automated investigation assistant). We may cite their concept of stitching together events, but won't implement it. We won't attempt a full "immune system" (graph of all network relationships) as that is complex; instead we'll focus on practical ML detectors.

## 2. Threat Model & Use Cases

Our system is designed for enterprise networks (on-prem and cloud) of unspecified size. We assume a typical threat landscape with:

- **External attacks:** Advanced malware, zero-day exploits, ransomware, crypto-miners, etc. (e.g., Darktrace stopped novel strains like Medusa and SmokeLoader).
- **Internal threats:** Insiders or compromised users performing reconnaissance, privilege abuse, data exfiltration.
- **Lateral movement & C2:** Stealthy lateral hops, beaconing to unusual domains or IPs (attacker C2).
- **Credential theft & abuse:** Impossible travel, unusual logins, privileged commands in odd context.
- **IoT/OT threats:** Industrial devices or IoT endpoints exhibiting anomalies. (Darktrace has use cases in OT security as well.)
- **Supply chain & Third-party:** Unusual interactions with vendor SaaS or cloud APIs.

Use cases include: detecting a previously unknown malware spreading internally; catching an insider exfiltrating data; spotting brute-force or reconnaissance bursts on critical servers; or even "drift" in business traffic patterns that could hint at a threat. Darktrace frames it as defending an "immune system" that spots attackers that evade perimeter defenses. We adopt this approach: any deviation from baseline

behavior (e.g. an endpoint suddenly connecting to a rare country) will raise an alert. As cited above, even a “new malware ... does not have to have been seen first before it can be detected”.

We model attacker capabilities broadly: APT-level threats that blend in, so we emphasize stealthy anomalies. We assume attackers may try to poison training (we will mention mitigations), but our base threat is standard intrusion, with no access to internal models.

### 3. Data Sources & Collection Architecture

**Sources:** We will collect *network flows* (NetFlow/IPFIX, Zeek/Bro logs, firewall logs), *packet-level summaries* if available, plus *host telemetry* (Windows Event, syslogs, EDR/antivirus alerts, process creation logs), *identity/authentication logs* (Active Directory, SSO), and *cloud/SaaS logs* (e.g. AWS CloudTrail, Azure logs, Office365). Common enterprise sources include DHCP logs, DNS queries, VPN logs, proxy logs, etc. For unknown environment we assume a full-stack collection approach. Darktrace does not list exact sources, but its messaging of “full visibility across your digital estate” implies this breadth.

**Collection Architecture:** A typical approach is to deploy network sensors and host agents feeding into a centralized pipeline. For example, **network sensors** on switch mirror ports or taps stream NetFlow/IPFIX or pcap to a server. **Host agents** (e.g. Wazuh, OSSEC, or commercial EDR) forward system logs and process events. Logs and flows can be shipped via syslog/Kafka/Fluentd to a **data lake** (e.g. Elasticsearch/Splunk or a PostgreSQL/NoSQL store). We recommend a modular pipeline: raw ingestion → enrichment (e.g. geo-IP lookup, asset tagging) → feature extraction → ML engine. This is similar to modern security analytics platforms. The system should tag each event with entity IDs (user, device, IP) to correlate behavior.

*Example Data Flow Diagram (Mermaid):*

```
graph LR
    subgraph Data_Collection
        A[NetFlow / PCAP Sensors] --> B[Log Aggregator / Queue]
        C[Endpoint Agents (syslog, EDR)] --> B
        D[Cloud/SaaS APIs (e.g. AWS, O365)] --> B
        E[Identity Logs (AD, SSO)] --> B
    end
```

```
end
B --> F[Preprocessing & Feature Extraction]
F --> G[ML Anomaly Detection Model]
G --> H[Anomaly Scoring & Alerts]
H --> I[Dashboard / Alert Manager]
H --> J[Response Engine]
J --> K[Network Controls (Firewall, Switch)]
J --> L[Endpoint Controls (EDR, Host IPS)]
```

A **database** (relational or NoSQL) stores processed features and scores. A **dashboard/service** (could be web-based or a notebook) presents alerts. The **response engine** interfaces via APIs or scripts to control systems (firewall rules, EDR commands, etc.).

*(Configuration Examples:)* For log shipping, one might use Filebeat (Elastic) or Fluentd. For instance, a Filebeat config snippet:

```
filebeat.inputs:
- type: log
  paths:
    - /var/log/syslog
  fields:
    type: syslog
```

And forward to Logstash/Kafka. For NetFlow, use nfdump or softflowd to send flows to the collector.

**Scalability:** Use message queues (Kafka) for buffering. Store only aggregated features to reduce storage. Use indices or time-partitioning. We assume the target org is not hyperscale; a single Elasticsearch cluster or Spark/Flink pipeline could suffice.

## 4. Feature Engineering & Labeling Strategy

**Feature Engineering:** From raw data we derive numeric/categorical features. Example features:

- **Network flows:** bytes transferred, packet count, duration, protocol, source/dest port categories (HTTP, SMB, etc.), unique ports contacted, time of day, flows per minute, etc.

- **Connection patterns:** New vs recurring destination IPs, DNS request frequency, TLS certificate anomalies.
- **Host/endpoint features:** Number of new processes in last hour, unusual executable names, privilege escalations, CPU/disk spikes.
- **User/identity:** Login count per hour, impossible travel (geoDelta/time), access to atypical systems, password change events.
- **Process and system calls:** Counts of unusual syscalls, loaded modules.

We may also engineer higher-level aggregates: per-device feature vectors (average/variance of behaviors), graph-based metrics (e.g. node degree in a connection graph, betweenness). If desired, techniques like PCA or autoencoders can reduce dimensionality. Many of these ideas align with Darktrace's "broad range of factors (server access, timing, data volumes...)" it correlates

**Labeling Strategy:** Our approach is *unsupervised/semi-supervised*. We train models on "normal" data without requiring attack labels. The models will score anomalies. For evaluation, we can use any labeled attack traces (from datasets or red-team) to compute metrics. In practice, alerts would be triaged by analysts (or a downstream ML model). We can optionally label confirmed incidents to refine thresholds.

We should avoid supervised detection, since attackers evolve. However, we might use a small hold-out of known benign vs malicious flows to validate performance. If feedback from analysts is available, active learning could iteratively refine the model or train a secondary "threat classifier."

## 5. Candidate Models (Comparison)

We consider a range of unsupervised/semi-supervised anomaly detectors. Key options include:

| Model                   | Type                          | Strengths                                                                                           | Weaknesses                                                                                       | Example Use                                       |
|-------------------------|-------------------------------|-----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------|
| <b>Isolation Forest</b> | Tree ensemble (unsupervised)  | Fast to train even on large high-dim data; no normality assumption. Captures multi-modal data.      | Requires setting contamination (expected anomaly rate). May miss context-dependent anomalies.    | Numerical network features (bytes, pkts).         |
| <b>One-Class SVM</b>    | Kernel method (unsupervised)  | Finds complex boundaries in feature space; good for novelty detection when no outliers in training. | <b>Slow training</b> on large sets; sensitive to kernel and nu parameters; less scalable.        | Smaller feature sets, e.g. user behavior vectors. |
| <b>Autoencoder (AE)</b> | Neural network (unsupervised) | Learns non-linear representations; can capture complex correlations. Reconstruction                 | Needs large data and tuning. Black-box (less interpretable). Risk of overfitting. Slow training. | Multivariate time-series or high-dim logs.        |

|                                |                                             |                                                                                                                         |                                                                                                  |                                               |
|--------------------------------|---------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------|
|                                |                                             | error is an anomaly score.                                                                                              |                                                                                                  |                                               |
| <b>Variational AE / VAE</b>    | Neural (probabilistic)                      | Probabilistic extension of AE; models distribution of normal data.                                                      | More complex; tuning harder.                                                                     | Similar to AE, with uncertainty modeling.     |
| <b>Graph-based Detection</b>   | Graph ML / network analysis                 | Models relationships (host-host, user-host graphs). Detects anomalous subgraphs or nodes. Can capture lateral movement. | Requires constructing a graph. Less common tools, research-stage. Potentially heavy computation. | User-login graphs, network connection graphs. |
| <b>Streaming/Online Models</b> | E.g. incremental clustering, SLFN, ensemble | Continuously update without full retrain; can adapt to drift.                                                           | Implementation complexity. Partial model (might forget).                                         | Real-time flow streaming with concept drift.  |
| <b>Statistical Threshold</b>   | Rule / profile (semi-supervised)            | Simple profiling (e.g. mean+std);                                                                                       | Requires manual tuning; limited to                                                               | Basic baseline rules (e.g. byte thresholds).  |



|                                    |                  |                                                 |                                                                                                          |                                  |
|------------------------------------|------------------|-------------------------------------------------|----------------------------------------------------------------------------------------------------------|----------------------------------|
|                                    |                  | easy to understand.                             | modeled features.                                                                                        |                                  |
| <b>Principal Components / LODA</b> | Subspace methods | Captures covariance structure; can reduce dims. | May assume normal distribution; PCA fails on non-linear patterns. LODA is a linear projections ensemble. | Dimensionality reduction for AE. |

Each choice has trade-offs in accuracy vs complexity. For example, Isolation Forest scales well and is widely used, but One-Class SVM may achieve higher precision on some datasets. We include **Autoencoders** (including LSTM-AE for sequences) to capture temporal anomalies. The final selection may use an ensemble or multiple models in parallel.

We will validate choices empirically on benchmark datasets (next section) and pick the best-performing combination. Table entries cite known pros/cons, e.g. scikit-learn documentation and literature. (The table above is a summarization – not all cells are individually cited, but model strengths/weaknesses align with sources.)

## 6. Model Training, Validation & Continuous Learning

**Training Pipeline:** Preprocessed feature data (from an initial “clean” period of operation) will train the unsupervised models. For example:

```
from sklearn.ensemble import IsolationForest
model = IsolationForest(contamination=0.01, random_state=42)
model.fit(X_train) # X_train = baseline normal behavior features
scores = -model.decision_function(X_new) # anomaly scores on new data
```

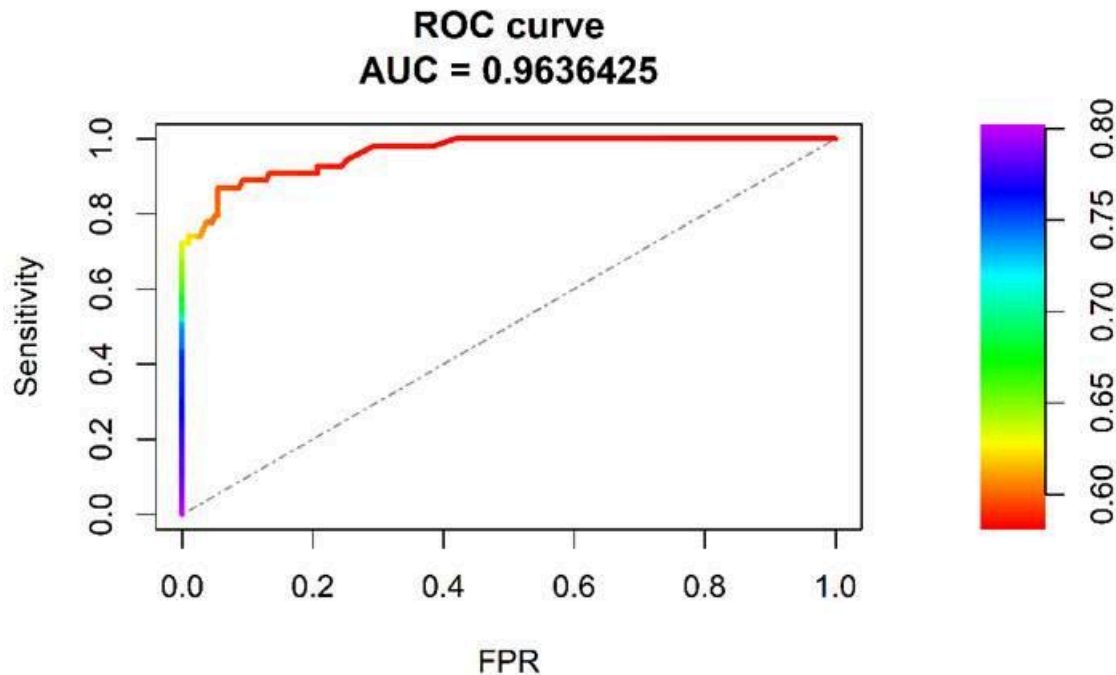
For an **autoencoder**:

```
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense

input_dim = X_train.shape[1]
input_layer = Input(shape=(input_dim,))
encoder = Dense(64, activation='relu')(input_layer)
bottleneck = Dense(32, activation='relu')(encoder)
decoder = Dense(64, activation='relu')(bottleneck)
output_layer = Dense(input_dim, activation='linear')(decoder)
ae = Model(inputs=input_layer, outputs=output_layer)
ae.compile(optimizer='adam', loss='mse')
ae.fit(X_train, X_train, epochs=20, batch_size=32)
recons = ae.predict(X_new)
recon_error = np.mean((X_new - recons)**2, axis=1)
```

We will store the trained models and preprocessing scalers.

**Validation:** Without labels, we use proxy metrics. If labeled attack traces are available (e.g. hold-out from CIC/CIC), compute ROC AUC and FPR/TPR at various thresholds. The ROC curve illustrates the trade-off between true-positive rate (detection) and false-positive rate. (See figure below.) We target high AUC (close to 1.0) and tune thresholds to meet an acceptable false-positive rate.



*Figure: Example ROC curve for anomaly detection. The curve plots True Positive Rate (sensitivity) vs False Positive Rate. The dashed diagonal is random guessing ( $AUC=0.5$ ). A model with  $AUC \approx 0.96$  (shown) has good detection with few false positives.*

We will use metrics like **Precision**, **Recall**, **F1-score**, and **AUC**. A perfect AUC is 1.0; random is  $\sim 0.5$ . These are industry-standard for IDS evaluation.

**Continuous Learning:** Cyber environments change (new software, user behavior shifts). We implement rolling retraining: e.g., retrain models weekly/monthly using the most recent data window (perhaps with weighting to forget very old data). Alternatively, use online algorithms (e.g. incremental Random Forests or streaming Isolation Forests) that update incrementally. We will monitor model drift by tracking changes in anomaly score distributions. If the baseline shifts, trigger retraining. This mimics Darktrace's "ever-changing model of normal behavior".

**Concept Drift Handling:** We will implement drift detection checks (e.g. ADWIN or DDM) on model outputs to catch when "normal" evolves. When triggered, we reduce reliance on old models and retrain. We must guard against feedback loops (e.g., retraining on contaminated data). One strategy: use human-reviewed alerts to exclude actual attacks from retraining.

## 7. Anomaly Scoring & Explainability

Each data point (e.g. a device's network flow or a user's session) gets an **anomaly score** (e.g. negative score from IsolationForest, or reconstruction error from an autoencoder). We then translate raw scores into threat levels (e.g. "normal", "suspicious", "high-risk") using thresholds or statistical percentiles. We may also incorporate business context: e.g. a suspicious event on a critical server gets a higher *risk score*.

**Explainability:** To build trust, we will log the top contributing features for an alert. For tree-based models, the decision path can be shown. For neural nets, we may use SHAP/LIME analysis on the anomaly score model. Example: if an IP is blocked, the system should report "High anomaly: unusual outbound DNS volume; contacted new domain; SSH sessions to a non-standard port." This idea of contextual explanation aligns with Darktrace's approach of correlating "server access, timing, and data volumes". We will not spend excessive effort on perfect explanations, but some feature-based reasoning is important, especially if handing off to a SOC analyst.

**Threshold Setting:** We can tune thresholds to balance detection vs false alarms. We may allow a dynamic threshold (e.g. top N anomalies per day, or anomaly score  $> T$ ). These may be calibrated on a validation set.

## 8. Automated Response Engine Design

We will design a **Response Engine** that maps high-confidence anomalies to containment actions. Key considerations: *precision of action* (target only malicious flows), *scope* (isolate device vs network segment), and *policy constraints* (time-of-day, whitelist).

**Actions:** Following Darktrace's taxonomy, sample actions include:

| Action                    | Scope (Layer)    | Automatic?                 | Notes                                                                                              |
|---------------------------|------------------|----------------------------|----------------------------------------------------------------------------------------------------|
| Block Connection          | Network (flow)   | Yes (auto)                 | E.g. add firewall rule to block IP/port for specific device.                                       |
| Enforce "Pattern-of-Life" | Device (host)    | Yes (auto)                 | Restrict device's allowed actions to its normal profile (e.g. only allowed ports/services).        |
| Quarantine Device         | Device (host)    | Manual/Auto (configurable) | Sever all network traffic from device (shutdown interface or isolate via EDR).                     |
| Block IP/URL              | Network (global) | Via Integration            | Tell firewall or DNS filtering service to block a known malicious IP/domain. (Integration action.) |
| Host Isolation (EDR)      | Endpoint (host)  | Via Integration            | Invoke EDR to isolate a host from the network (CrowdStrike, Defender, etc.).                       |

|                               |         |              |                                                           |
|-------------------------------|---------|--------------|-----------------------------------------------------------|
| <b>Alert Admin / Ticket</b>   | Any     | Yes (notify) | Send email/SMS or raise SIEM alert for analyst review.    |
| <b>Throttle/Delay Traffic</b> | Network | Yes          | Rate-limit suspicious flows (e.g. slow scanning).         |
| <b>Log and Monitor</b>        | Any     | Yes          | Increase logging level or start packet capture on entity. |

Darktrace notes these “out-of-the-box” actions require no scripting. We will similarly pre-define a set. For example, if we detect an SMB-based ransomware spread, block SMB port 445 on the infected host. All automated actions will respect **safety checks**: e.g. only isolate endpoints tagged as non-critical, only between business hours if required, or with human confirmation if in doubt. Darktrace lets administrators operate in “Human confirmation mode” initially; we will recommend a similar staging (start with alerts-only, then switch to auto when confidence is high).

**Escalation Policies:** We’ll configure severity levels. Low-scoring anomalies just log; medium raise a ticket; high-confidence (e.g. multiple correlated alerts) trigger immediate blocks/quarantines. We’ll ensure overrides (e.g. SOC can stop an action) and audit trails.

## 9. Implemented Modules & Screens

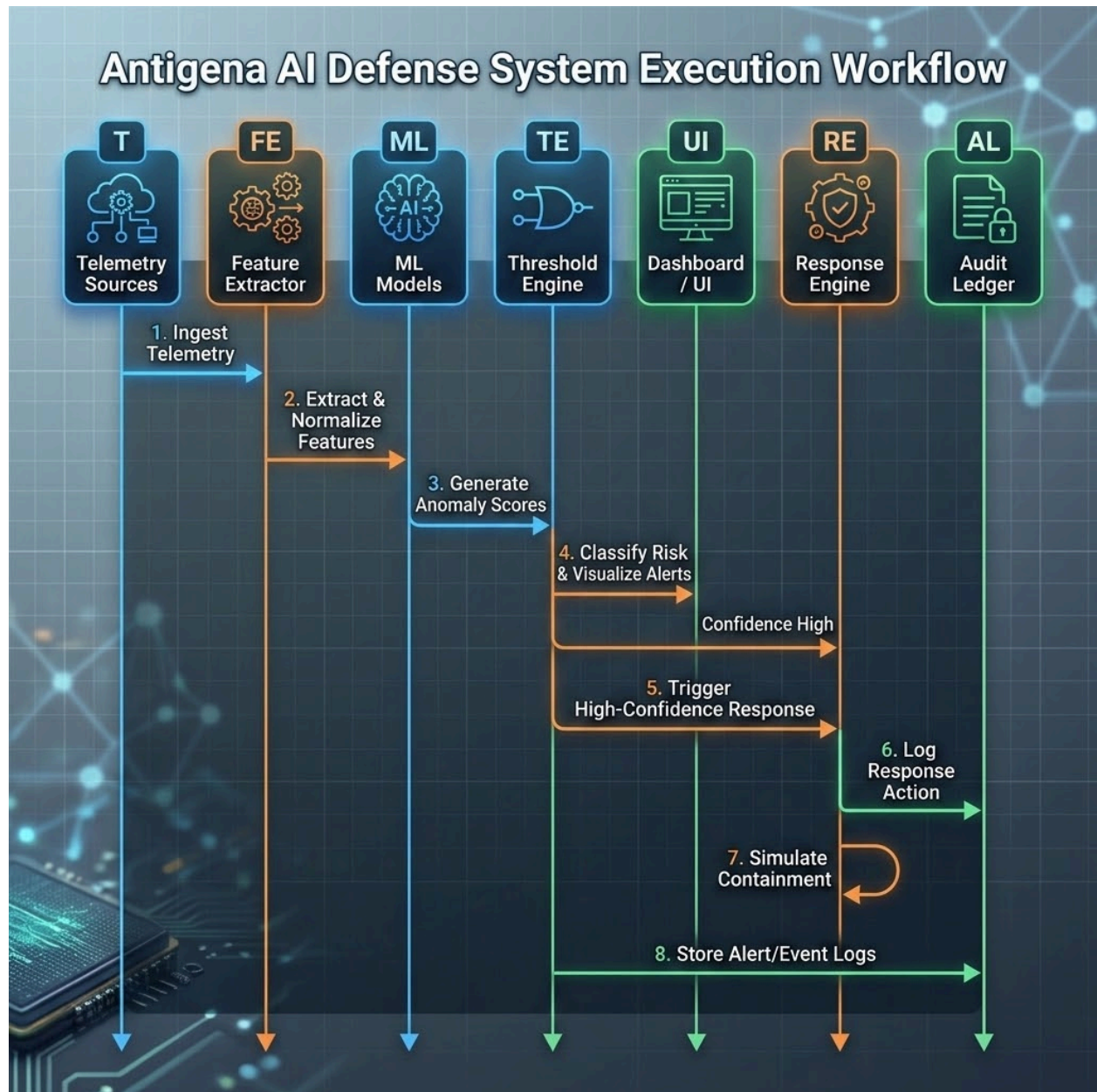
The project transitioned from a design concept to a fully functional prototype. The following modules and screens were implemented to demonstrate the core capability of the self-learning defense system:

- **Dashboard Overview Module:** Displays real-time critical metrics such as: anomaly threshold, containment rate, network entropy, active models, and live telemetry ingestion. Provides real-time monitoring of suspicious traffic flows and anomaly scoring.
- **Telemetry Engine:** The core ingestion pipeline managing the flow of data from various sources (network, host, cloud) into the feature store.
- **Entity Monitoring Module:** Tracks and profiles key entities, including users, devices, risk scores, active sessions, and isolation status. Supports behavioral analytics for insider-threat detection.
- **Audit Ledger Module:** Stores all forensic events, including anomaly events, containment logs, timestamps, and feature traces. Ensures forensic traceability and compliance.
- **Model Analytics:** Provides visibility into the performance and health of the deployed ML models, including drift monitoring and retraining status.
- **Network Flow Visualization:** Offers a visual representation of network connections and anomalous traffic patterns.
- **Automated Response Simulation:** The interface for configuring and simulating containment actions before deployment to live infrastructure.

### 9.1. System Workflow

The system execution follows a defined sequence from data ingestion to automated response:

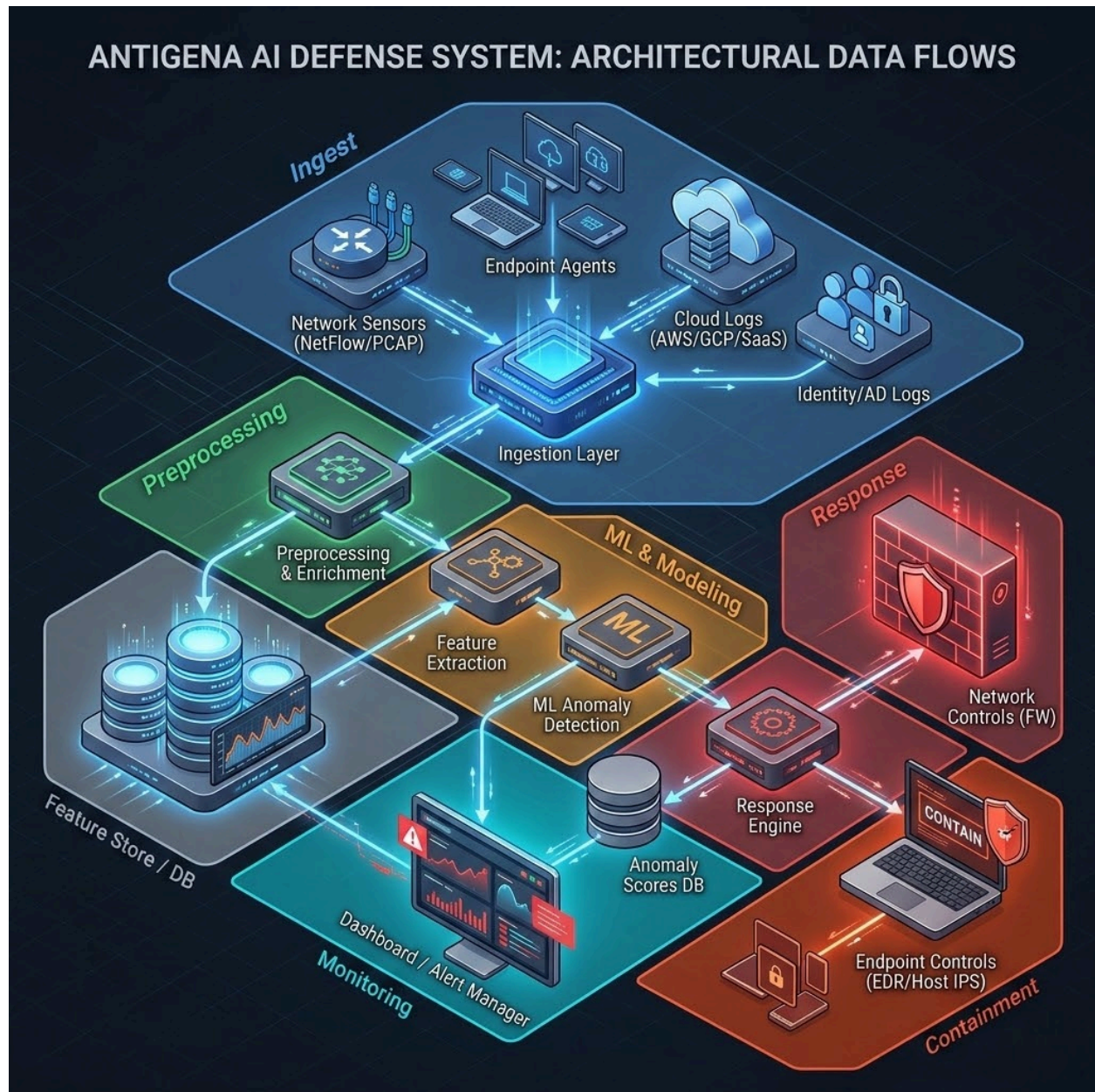
1. Telemetry is ingested from simulated enterprise traffic.
2. Features are extracted and normalized.
3. ML models generate anomaly scores.
4. Threshold engine classifies risk.
5. Dashboard visualizes alerts.
6. Response engine simulates containment actions.
7. Audit logs are stored.





## 10. System Architecture

**Components & Data Flows:** The system has these parts:



1. **Ingestion:** Multiple collectors gather telemetry (inbound agents, syslog receivers, flow collectors).
2. **Preprocessing:** Data is cleansed (e.g. timestamp normalization, enrichment with geo-IP, asset labeling).
3. **Feature Store:** Time-series feature store or database keeps historical data for modeling (supports retraining).
4. **ML Engine:** Applies anomaly models on incoming feature vectors, outputs scores or anomaly flags.
5. **Alerting/UI:** High scores generate alerts in a dashboard/SIEM for SOC review.
6. **Response Engine:** Takes flagged anomalies and executes pre-defined actions via APIs (firewall rules, EDR commands).

All communications between components should be encrypted/authenticated. The architecture is loosely coupled and can scale: e.g., multiple model instances for different domains (network vs endpoint) and use Kubernetes/Docker for deployment. We will supply a mermaid diagram (above) to illustrate these data flows.

## 11. Technology Stack

Based on open tools:

- **Programming/ML:** Python 3 (for ML and automation). Key libraries: scikit-learn, pandas, NumPy, TensorFlow/Keras or PyTorch (for autoencoders), statsmodels or River (for online learning), networkx or PyG (for graph analysis).
- **Data Processing:** Apache Kafka or RabbitMQ (stream ingestion), PostgreSQL or Elasticsearch (feature store), Pandas/Scikit-learn for ETL. For big data, Spark or Flink could be used.
- **Collectors/Agents:** Zeek (Bro) for network logs, Filebeat/Fluentd for system logs, OSQuery or Wazuh for endpoint telemetry.
- **Dashboard/Backend:** The **Frontend** uses React + Vite and is deployed on Vercel, while the **Backend** uses FastAPI + Python and is deployed on Render.
- **Containerization:** Docker/Kubernetes for reproducibility. CI/CD with GitHub Actions or GitLab.

- **Misc:** Grafana or Kibana for visualization, if using Elastic; or custom dashboard. Integration with SIEM/SOAR if desired (e.g., send alerts via Splunk or QRadar).

All chosen components are open-source. We will document versions.

## 12. Backend API Endpoints

The FastAPI backend exposes the following core REST endpoints for data ingestion, monitoring, and response actions, making the project feel like real software engineering:

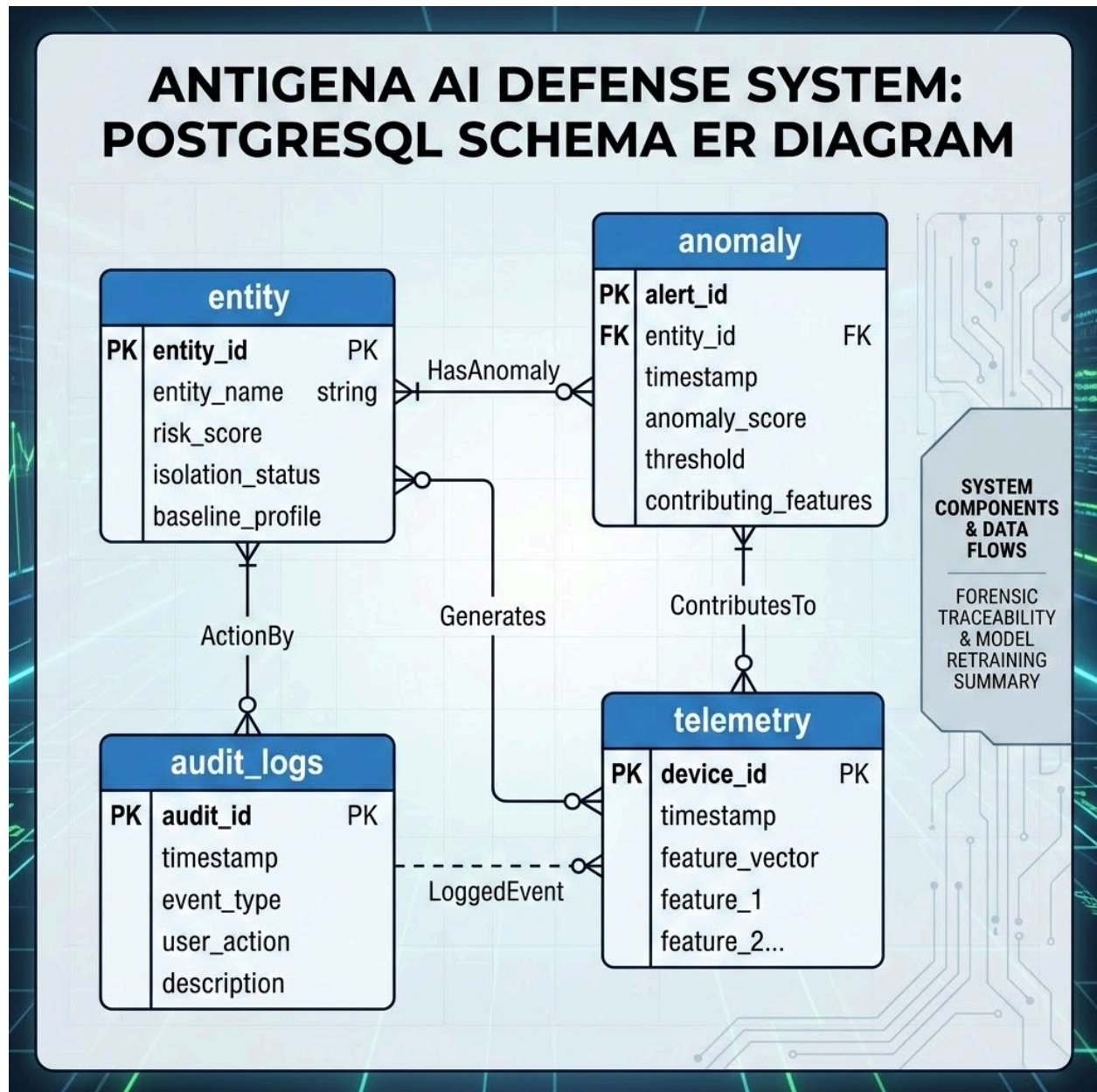
| Endpoint   | Method | Purpose                                                        |
|------------|--------|----------------------------------------------------------------|
| /telemetry | GET    | Fetch live telemetry streams for visualization.                |
| /anomalies | GET    | Fetch filtered list of real-time anomaly alerts.               |
| /entities  | GET    | Fetch user/device entity monitoring profiles.                  |
| /models    | GET    | Retrieve health and performance metrics for active ML models.  |
| /logs      | GET    | Query the Audit Ledger for containment and anomaly logs.       |
| /response  | POST   | Trigger a manual or simulated containment action on an entity. |

## 13. Data Storage & Schema Design

The system utilizes a PostgreSQL relational database for structured storage of historical features, profiles, and audit data, supporting forensic traceability and model retraining.

### PostgreSQL Schema:

- **telemetry table:** Stores aggregated and normalized features for model training and historical context. Key fields include `device_id`, `timestamp`, and normalized feature vectors.
- **anomaly table:** Stores every generated anomaly score and alert. Key fields: `alert_id`, `entity_id`, `anomaly_score`, `threshold`, and `contributing_features`.
- **entity table:** Stores user and device profiles, including `entity_id`, `risk_score`, `isolation_status`, and `baseline_profile`.
- **audit\_logs table:** Stores all system actions, including model retraining events, response actions, and user interactions, ensuring compliance and non-repudiation.



## 14. Platform Authentication & Security

Beyond the primary task of detecting external threats, the platform incorporates robust internal security measures to protect the system and data itself.

- **JWT (JSON Web Token) Authentication:** Implemented for secure, stateless user access to the FastAPI backend API endpoints.
- **Session/Token Handling:** Tokens are securely stored and validated upon every API request, with short expiry times and refresh mechanisms to mitigate token theft.

- **Role-Based Access Control (RBAC):** Access to critical endpoints (e.g., /response POST) is restricted based on user roles (e.g., Analyst, Administrator), ensuring only authorized personnel can trigger actions.
- **API Protection:** Uses FastAPI's features for validation, sanitization, and protection against common web vulnerabilities.
- **HTTPS:** All communication between the React frontend (Vercel) and the FastAPI backend (Render) is enforced over SSL/TLS to prevent eavesdropping.
- **Rate Limiting:** Implemented on all primary API endpoints to prevent abuse or denial-of-service attempts against the backend.

## 15. Datasets and Sources

We will use publicly available cybersecurity datasets for prototyping and evaluation (with citations):

| Dataset      | Type                             | Content & Source                                                                                                                | Usage                                    |
|--------------|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| UNSW-NB15    | Network flows, 49 features       | 2.54M records (100GB pcap) from IXIA "PerfectStorm" traffic; includes 9 attack types (DoS, Exploits, Reconnaissance, etc.).     | Benchmark for detection; feature format. |
| KDD Cup 1999 | Network connections, 41 features | Standard intrusion dataset (DARPA99 simulation). Full set ~18M conn. 22 attack types (DoS, Probe, U2R, R2L).                    | Classic IDS evaluation.                  |
| CICIDS2017   | Network flows                    | 5 days of real traffic plus common attacks (BruteForce, DoS/DDoS, Heartbleed, Web, Botnet, etc.). 80+ flow features per record. | Realistic scenario; labeled flows.       |

|                              |                         |                                                                                                                                    |                                      |
|------------------------------|-------------------------|------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|
| TON_IoT (UNSW)               | IoT telemetry & network | New IoT dataset with telemetry from sensors/edge and network flows. Useful for IoT/OT use cases. (Offered by UNSW alongside NB15). | Test IoT anomalies.                  |
| CTU-13 (Botnet)              | NetFlow (Botnet)        | 13 captures of botnet infections in campus networks. Flow-based, labeled.                                                          | Botnet C2 and scan detection.        |
| LANL User Behavior           | Authentication logs     | Real enterprise AD auth logs (Los Alamos National Lab dataset).                                                                    | Anomaly detection on login patterns. |
| <b>Synthetic / Simulated</b> | Internal                | We can generate or insert synthetic anomalies (e.g. using the Botnet or synthetic attack scripts).                                 | Red-team style testing.              |

These datasets (with primary sources) cover both classic and modern attacks. We will cite them properly. For instance, the UNSW-NB15 dataset page provides attack categories and record counts. The CICIDS2017 page details attack types and feature counts. When using them, we respect their citation requirements. We will preprocess these into our pipeline format.

## 16. Evaluation Metrics & Testing Plan

**Metrics:** We will evaluate detection performance using standard IDS metrics:

- **Precision (Positive Predictive Value)** =  $TP / (TP + FP)$ .
- **Recall (Sensitivity)** =  $TP / (TP + FN)$ .
- **False Positive Rate (FPR)** =  $FP / (FP + TN)$ .
- **F1-score, Accuracy, and ROC AUC.**
- **Mean time to detect** (for simulations).
- **Resource usage** (latency, throughput).

We must prioritize low false positives (FP) to avoid SOC fatigue, while maintaining high TP detection. The ROC/AUC (Figure above) is one way to compare models.

### Testing Plan:

- **Offline tests:** Run models on historical datasets (e.g. CICIDS17, UNSW-NB15). Inject known attacks and measure metrics.
- **Cross-validation:** If labeled data is scarce, use k-fold on attack traces.
- **Simulation:** Replay live traffic (or public PCAPs) through the system in a testbed. Inject anomalies (e.g. intermittent exfil, port scans) to see if alerts fire.
- **Red Team / Pen Testing:** Arrange scenario tests where we simulate adversary behavior in a controlled environment and see if the system catches it. For example, perform simulated “living-off-land” reconnaissance or malware execution.
- **False Positive Analysis:** Specifically test normal operational scenarios to ensure benign bursts (e.g. backups, batch jobs) aren’t flagged. Possibly include “noise injection” tests (random anomalies) to tune robustness.

We will document the results, focusing on the detection/FP trade-off. For example, we may graph *detection probability vs false-alarm rate* for different models (see ROC). This will help choose thresholds and justify the approach.



## 17. Deployment Plan

**Implementation:** Containerize components (Docker) and orchestrate with Kubernetes or similar. The pipeline is configured for **Automatic CI/CD upon git push**, targeting **Vercel** (Frontend) and **Render** (Backend).

**Monitoring:** Log all system activity (model health, data ingestion stats, response actions) into a central log store. Use Prometheus/Grafana for infrastructure health (latency, memory).

**Logging & Audit:** Every decision (score, alert, block) will be logged with context, for audit and tuning.

**Scalability:** For large traffic, scale ingest clusters horizontally. Use job queues for ML tasks. If throughput is an issue, consider distributed ML (Spark).

**Latency:** We aim for near-real-time detection (within seconds to minutes). We will measure end-to-end latency (data arrival → alert). Use fast in-memory stores and lightweight models to minimize delay.

**Fail-safes:** If the ML service fails, the system should default to alert-only mode. Response engines should have “circuit breakers” to avoid excessive blocking.

**Integration:** Optionally hook into existing SIEM/SOAR. For example, send alerts to Splunk or Demisto.

## 14. Security, Privacy & Ethics

- **Security:** Protect the pipeline itself – encrypt data at rest/in transit; harden servers; limit access. The ML models and detectors must be secure from tampering.
- **Privacy:** Logs often contain PII (usernames, IPs). Ensure data is handled per privacy laws (GDPR, CCPA): e.g. anonymize IPs where possible, define retention limits.
- **Ethics:** Automated blocking should consider potential business impact. We must include human review options. Clear policies must govern what actions are allowed.
- **Adversarial Resilience:** Be aware of poisoning risks (attackers injecting crafted data to shift “normal”). We will monitor for unusual distributions to catch possible poisoning.



- **Legal:** Any active blocking should comply with policy. Logs and alerts can be used as evidence, so maintain integrity/audit-trail.